PUCPR
GRUPO MARISTA

Complexidade de Algoritmos

Aula Inaugural

Felipe A. Przysiada

PUCPR

Sobre o professor

- Engenheiro da Computação;
- Pós em Sistemas Embarcados;
- Mestre em Engenharia Elétrica e Informática Industrial, área de concentração Engenharia de Automação e Sistemas;

Resultados de Aprendizado (RA)

- Emprega técnicas de contagem, soma e prova matemática na análise de algoritmos interativos para problemas de computação, de acordo com o contexto de aplicação e preceitos éticos;
- Emprega técnicas de contagem, soma, resolução de recorrência, prova matemática na análise de algoritmos recursivos e randomizados para problemas de computação, de acordo com as características do problema a resolver e contexto; (Recuperação)
- Seleciona técnicas e algoritmos para resolver de forma aproximada problemas intratáveis, de acordo com as características do problema e contexto, e preceitos éticos e legais;

Ementa

- O estudante aplica conceitos e métodos da matemática discreta na análise de complexidade de algoritmos iterativos, recursivos e randomizados;
- Ao término da disciplina, é capaz de projetar algoritmos eficientes e expressar suas complexidades, utilizando métodos de contagem, resolução de recorrência e notação assintótica de funções de complexidade;
- Os estudantes trabalham em equipes para verificar os benefícios da análise de algoritmos por meio de experimentos realistas com vistas as análises de tempo de processamento e uso de memória. Para frequentar a disciplina é necessário conhecimento em raciocínio algorítmico, estrutura de dados e matemática discreta;

Metodologia

- Uma mistura entre conceituação e aplicações;
- Note que cabe a você a sua equipe utilizar cada ferramenta (ou não) no projeto que será desenvolvido;
- Importante: A ideia é desenvolver autonomia, uma característica chave para qualquer engenheiro;
- Para isso iremos trabalhar com:
  - Aula expositiva dialogada;
  - Ensino baseado em problemas;
  - Desenvolvimento de projetos;

Carga Horária

- Disciplina de 80 horas/aula;
- 60 horas/aula presencial + 20 horas/aula TDE;
- 4 horas / aula por semana:
  - Quarta - 19:45 / 23:00;

Avaliações

Atividade	RA1	RA2	RA3
Avaliação 1	60%		
Projeto Colaborativo 1	40%	40%	
Avaliação 2		60%	
Avaliação 3			100%
TOTAL	100%	100%	100%

PUCPR
GRUPO MARISTA

Equipes

- Vocês devem criar uma equipe no Canvas com 4 estudantes (sem exceções);
- FAQ:
  - Posso fazer sozinho? Não
  - Posso fazer em dupla? Não
  - Posso fazer um grupo com cinco? Não
- Todos os estudantes devem estar designados até o final da aula;
- Alunos que não estiverem designados vão ser designados randomicamente;

Regras de Convivência

- Celulares? Modo Silencioso;
- Banheiro? Vai;
- Horário de Aula? Deve ser cumprido;
- Aulas e atividades? Todas no AVA;
- Brincadeiras são bem-vindas, no momento certo;
- Conversa no momento errado atrapalha todo mundo;
- Perguntas são bem-vindas;
- No pain No gain = Trabalho copiado, trabalho zerado;

Considerações Pontuais

- Durante as aulas e o projeto, você e sua equipe deverão demonstrar AUTONOMIA. Se houver algo sobre o qual não falamos, mas que você considere adequado: vá em frente. Experimente. Vá além. Não importa se você está buscando uma posição acadêmica, industrial ou orientada ao mercado; será necessário conduzir pesquisas em direção ao seu objetivo e desenvolver suas habilidades.
- O professor não deve ser enxergado como uma figura que ensina. O real papel do professor é atuar como um facilitador e orientador, que ajuda o aluno a aprender. Na verdade, não é o professor que ensina, mas sim o aluno que aprende!
- Uma sala de aula é como uma orquestra: para que as coisas fluam com harmonia, deve haver interação, cooperação e colaboração, e cada um deve cumprir o seu papel!
- Todos aprendem, todos aprendem o tempo todo, cada um aprende de um jeito;

Introdução







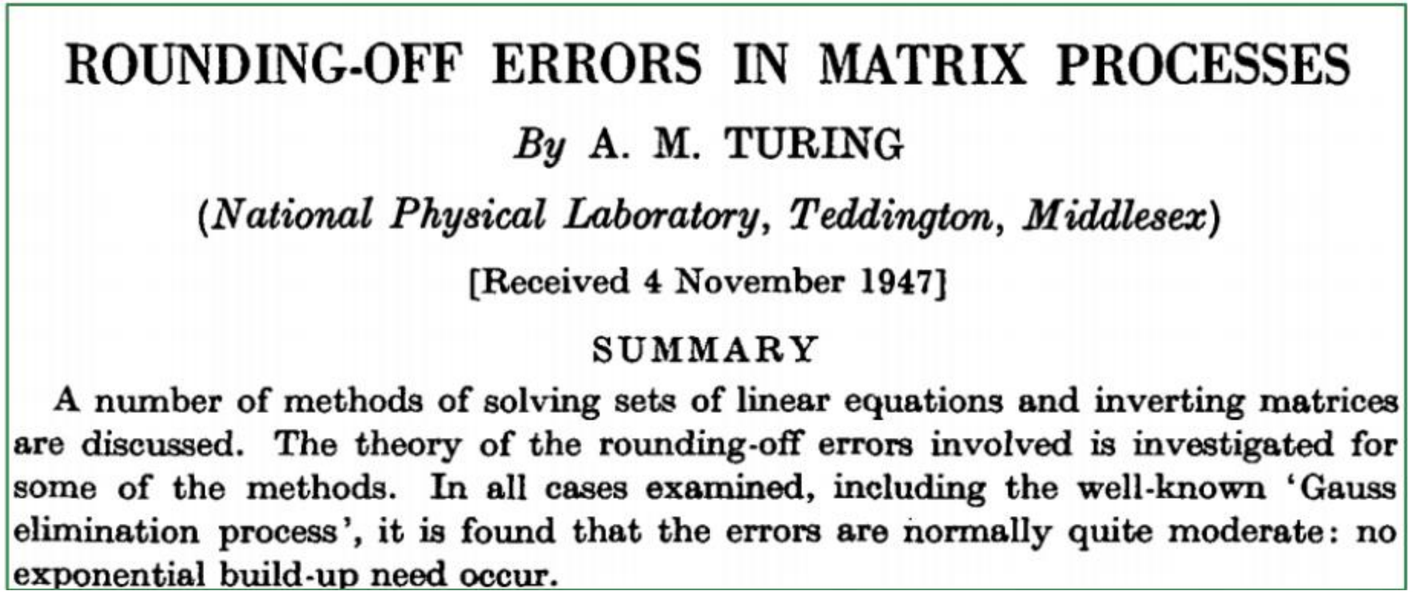
Como saber se o algoritmo é bom?

- Intuitivamente:
  - testar se o algoritmo funciona;
  - ver se é rápido;
    - como saber se o algoritmo funciona?
      - implementação cuidadosa e testar várias entradas;
  - como saber se ele é rápido?
    - testar entradas grandes e variadas e ver se executa bem;

PUCPR
GRUPO MARISTA

Como saber se o algoritmo é bom?

- "É conveniente ter uma medida da quantidade de trabalho envolvida em um processo de computação, mesmo que seja muito bruta ... Podemos, por exemplo, contar o número de adições, subtrações, multiplicações, divisões, gravações de números e extrações de figuras de tabelas." - Alan Turing, 1947



ROUNDING-OFF ERRORS IN MATRIX PROCESSES

By A. M. TURING

(*National Physical Laboratory, Teddington, Middlesex*)

[Received 4 November 1947]

SUMMARY

A number of methods of solving sets of linear equations and inverting matrices are discussed. The theory of the rounding-off errors involved is investigated for some of the methods. In all cases examined, including the well-known 'Gauss elimination process', it is found that the errors are normally quite moderate: no exponential build-up need occur.

PUCPR
GRUPO MARISTA

Pergunta

- Qual é o tempo de execução deste algoritmo?

```
boolean algoritmo (char[] v) {
    int n = v.length();
    for (int i = 0; i < n; i++)
        if (v[i] != v[n-i-1])
            return false;
    return true;
}
```

O(1)

O(log n)

$O(n^{1/2})$

O(n)

O(n log n)

$O(n^2)$

$O(n^2 \log n)$

$O(2^n)$

PUCPR GRUPO MARISTA

Análise de Algoritmos

- Nesta altura do campeonato, você já deve ter visto ao menos uma análise de algoritmo (mesmo que superficialmente);
- Como provavelmente você viu análise de algoritmo?
  - CASO 1: há dois laços aninhados, indo até n. Portanto, o algoritmo é $O(n^2)$;
  - CASO 2: contar uma operação específica do algoritmo (comparações, trocas, ...);

PUCPR
GRUPO MARISTA

Selection Sort

- ideia: seleciona o 1º menor elemento, troca com 1ª posição, seleciona o 2º menor elemento, troca com a 2ª posição, ...

vetor original:

1	2	3	4	5	6	7	8	9	10
15	12	27	23	7	2	0	18	19	21

execução:

1	2	3	4	5	6	7	8	9	10
0	12	27	23	7	2	**15**	18	19	21
1	2	3	4	5	6	7	8	9	10
0	**2**	27	23	7	**12**	15	18	19	21
0	2	**7**	23	**27**	12	15	18	19	21
0	2	7	**12**	27	**23**	15	18	19	21
0	2	7	12	**15**	23	**27**	18	19	21
0	2	7	12	15	**18**	27	**23**	19	21
0	2	7	12	15	18	**19**	23	**27**	21
0	2	7	12	15	18	19	**21**	27	**23**
0	2	7	12	15	18	19	21	**23**	**27**

PUCPR GRUPO MARISTA

Selection Sort

```
SelectionSort(A[1...n])
    para i ← 1 até n - 1
        menor ← i
        para j ← i + 1 até n
            se A[menor] > A[j]
                menor ← j
        Troca(A, i, menor)
```

```
Troca(A, i, j)
    temp ← A[i]
    A[i] ← A[j]
    A[j] ← temp
```


PUCPR
GRUPO MARISTA

Análise - Selection Sort

- Exemplo de análise contando as comparações;

```
SelectionSort(A[1...n])
    para i ← 1 até n - 1
        menor ← i
        para j ← i + 1 até n
            se A[menor] > A[j]
                menor ← j
        Troca(A, i, menor)
```

quando $i = 1$:
- j vai de 2 até n
- `if` é executado $n - 1$ vezes

quando $i = 2$:
- j vai de 3 até n
- `if` é executado $n - 2$ vezes

quando $i = 3$:
- j vai de 4 até n
- `if` é executado $n - 3$ vezes

$\vdots$

quando $i = n - 2$:
- j vai de $n - 1$ até n
- `if` é executado 2 vezes

quando $i = n - 1$:
- j vai de n até n
- `if` é executado 1 vez

PUCPR GRUPO MARISTA

Análise - Selection Sort

- contando as comparações do selection sort:
- $S = n - 1 + n - 2 + n - 3 + \cdots + 2 + 1$

$$= \sum_{i=1}^{n-1} i$$



Carl Friedrich Gauss
(1777 - 1855)

$$= \frac{n(n-1)}{2}$$

$$\approx \frac{n^2}{2} \approx \boxed{n^2}$$

PUCPR
GRUPO MARISTA

Objetivo: Comparar Algoritmos

- para a ordenação:
  - algoritmos “ingênuos”: $n2$
  - algoritmos “espertos”: n log2 n

n	n^2	$n \log_2 n$
10	100	320
100	10000	650
1000	1 milhão	10000
1 milhão	1 trilhão	20 milhões
1 bilhão	10^9 bilhões	30 bilhões

Parênteses: Velocidade

- medida popular, porém imprecisa:
  - núm. de instruções/segundo;
- atualmente: alguns bilhões de instruções/segundo

n	ingênuo	esperto
10	–	–
100	0,00001 s	–
1000	0,001 s	0,00001 s
1 milhão	16,6 min	0,2 s
1 bilhão	31,7 anos	30 s

Parênteses: Velocidade

- ENIAC: ≈ 5 mil instr/seg;
- Intel I9: ≈ 5 bilhões instr/seg;

- Intel I9 é 1 milhão de vezes mais rápido que ENIAC;

- n grande: algoritmo bom no ENIAC é melhor que um algoritmo ruim no I9



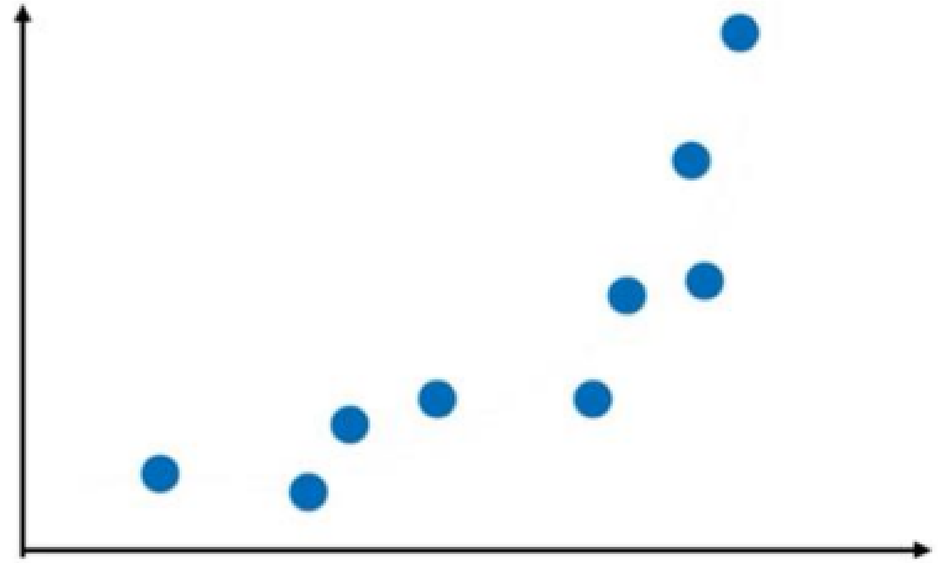
vamos tentar uma forma alternativa da análise...

Exemplo: Ordenação

- Executou para vetor de tam n = 100, levou 1s;
- só isso não traz muita informação... vou executar para outros tamanhos:
- Executou para vetor de tam n=200, levou 2s;
- Executou para vetor de tam n=300, levou 4s;
- Executou para vetor de tam n=400, levou 7s;
- Executou para vetor de tam n=500, levou 17s;
- Executou para vetor de tam n=600, levou 30s;

Exemplo: Ordenação

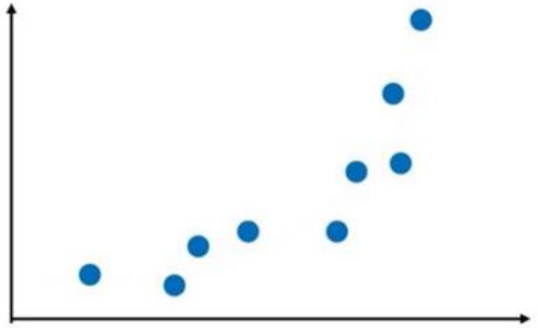
plota o gráfico:



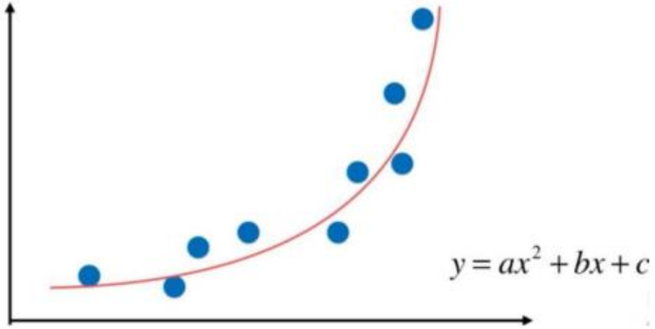
PUCPR
GRUPO MARISTA

Exemplo: Ordenação

plota o gráfico:



faz uma regressão:



- Encontrou função quadrática! Portanto, é um algoritmo de ordenação ≈ n^2

Isso é tudo?

- Há muitas outras coisas a serem estudadas:
  - Para mesmo n, há diferença no tempo de execução de entradas diferentes?
  - Qual entrada faz o algoritmo ter a pior execução?
  - Qual entrada faz o algoritmo ter a melhor execução?
  - As entradas ruins (boas) são raras ou comuns?
  - Qual o trecho do algoritmo faz as entrada ruins executarem mal?
  - Se eu alterar esse trecho do algoritmo, quão melhor ele executará?
  - Existe alguma entrada que o algoritmo devolve resposta errada?
  - Qual é o ponto fraco do problema? Como bolar um algoritmo para isso?

PUCPR
GRUPO MARISTA

Problema Computacional

- um problema computacional é um problema formulado em termos de
  - dado . . . obtenha . . .
  - ou
  - entrada: . . . saída: . . .
  - ou
  - instância: . . . resposta: . . .

PUCPR
GRUPO MARISTA

Problema Mal-Definidos

- problema: detectar comunidades em redes sociais;
  - o que é uma comunidade?
- solução: definir precisamente!
  - 1. método do corte mínimo;
  - 2. método de agrupamento hierárquico;
  - 3. algoritmo de Girvan–Newman;
  - 4. maximização de modularidade;
  - 5. baseado em cliques;

PUCPR
GRUPO MARISTA

Problema Mal-Definidos

- alguns casos são ainda mais problemáticos...
- problema: classificar uma imagem como cão ou gato
  - o que é uma imagem de cão?
  - o que é uma imagem de gato?
- solução: definições “precisas” (entre aspas)
  - 1. redes neurais
  - 2. classificadores lineares
- ainda não se sabe exatamente porque eles funcionam!!!



PUCPR
GRUPO MARISTA

Algoritmos

- a solução de um problema computacional é chamado de algoritmo;

- um algoritmo é:
  - a descrição de uma computação (passo-a-passo);
  - para cada instância do problema, resulta numa resposta;

Algoritmos - Descrição

- vamos descrever algoritmos usando pseudo-código;

  - parece código, mas não é!
  - é mais simples (omite detalhes específicos usados para o computador entender (declarar variáveis, delimitadores, ...);
  - a maioria dos livros de algoritmos usa pseudo-código;

PUCPR
GRUPO MARISTA

Algoritmos

- Note que não é preciso um computador para executar a computação descrita pelo algoritmo!
- Uma pessoa, um animal treinado, ou um dispositivo mecânico movido a querosene são capazes de executá-lo;

- “Ciência da computação está relacionada com o computador assim como a Astronomia com o telescópio, a Biologia com o microscópio, ou a Química com os tubos de ensaio. A Ciência não estuda ferramentas, mas o que fazemos e o que descobrimos com elas.” - Edsger Dijkstra

Algoritmos

- já vimos que:
  - Computador não é necessário para fazer computação;
  - Linguagens de programação não são necessárias para descrever algoritmos;

- Nossa abordagem é mais “purista”, não dependente da tecnologia;

- Essa abordagem purista é o padrão de abordagem na área de algoritmos;

Conheça o Adversário

- Um mundo cruel:
  - entrada diabolicamente planejada;
  - abordagem de pior caso;
  - seremos honestos: se algoritmo é bom, nenhuma entrada tem mal desempenho;

abordagens alternativas:

- **melhor caso** } ignora o adversário
- **caso médio** }
- **caso esperado** } engana o adversário

PUCPR GRUPO MARISTA

Como analisar: Tamanho Importa



- Buscar em bilhões de palavras > buscar em 2 palavras
- Análise depende do tamanho n da entrada
- Tamanho n é:
  - geralmente: #bits da codificação binária da entrada;
  - às vezes: explicitamente definido como sendo outra coisa;

- GRANDE IDEIA # 1: ANÁLISE É UMA FUNÇÃO EM n

Como analisar: Valores Importam

- Diferentes entradas tem diferentes valores!
  - Para entradas de (mesmo) tamanho n:
    - Algumas levam 2 passos;
    - Algumas levam bilhões de passos;

- GRANDE IDEIA # 2: ANÁLISE PELA PIOR ENTRADA DE TAMANHO n

Exemplo 1

```
void somavet (int v[n], int k) {
   int i;
   k=0;
   for (i=0; i<n; i++)
      k = k + v[i]; }
```

T(n) = 6n + 3

PUCPR
GRUPO MARISTA

Exemplo 2

```
void somavet (int v[n], int k) {
    int i;
    k=0;
    for (i=0; i<=n; i++)
        k = k + v[i]; }
```

$T(n) = 7n + 3$

PUCPR
GRUPO MARISTA

Exemplo 3

```
void somavet (int v[n], int k) {
    int i;
    k=0;
    for (i=0; i<n; i++)
        for (i=0; i<n; i++)
            k = k + v[i]; }
```

$T(n) = 6n^2 + 4n + 3$

PUCPR
GRUPO MARISTA

Exemplo 4

```
int localiza (int v[n], int x) {
    int i;
    for (i=0; i<n; i++)
        if (x==v[i])
            return i;
    return -1; }
```

A complexidade é sempre a mesma?

PUCPR
GRUPO MARISTA

Isso é tudo pessoal

Questões?

SCIENTIA, VITA ET FIDES

PUCPR